

Mark Samuel

Richmond Hill, Ontario, Canada

☎ +1 647 325 5584 ✉ mark.samuel@mail.utoronto.ca 🔗 linkedin.com/in/ms-ee

Education

University of Toronto

Expected May 2028

B.A.Sc. in Electrical Engineering — Minors: AI/ML & Robotics/Mechatronics

Toronto, ON

- Relevant coursework: Computer Architecture, Digital Electronics, Hardware Design, Computer Hardware, Data Structures & Algorithms, Machine Learning, Artificial Intelligence, C/C++ Programming.

Technical Skills

Languages & Tools: Verilog, Tcl, Python, C, C++, Bash, SQL, JavaScript, HTML, CSS; Perforce, Git, Linux, LSF, React, FastAPI, Postgres, FPGA (Quartus, DE1-SoC), Altium, Matlab.

Signoff, Synthesis & PPA: RTL lint (SpyGlass/Leda), CDC (Questa CDC), simulation (VCS, ModelSim), logic synthesis (Design Compiler), netlist review, SRAM/memory-compiler flows, ECC wrappers, PPA analysis.

Digital & Embedded: PCIe (TL/PL), RISC-V ISA/Assembly pipelining, memory hierarchy, CDC synchronizers, async FIFOs, HW/SW co-design, STM32, ESP32, Arduino, PWM, I²C, SPI, UART, BLE HID, Testbench equipment.

Experience

Silicon Implementation & Integration Intern — PCIe Subsystem

May 2026 – Present

AMD (Advanced Micro Devices)

Toronto, ON

- Own the team's Tcl async-FIFO extraction script using RTL and build-time analysis to deliver reference lists to be checked for post-synthesis netlist equivalence and read/write-pointer race conditions and gray-code crossings.
- Triage RTL lint, CDC, and VCS sanity violations at the BIF core & Subsystem level, driving root-cause resolution directly with designers and frequently proposing the fix.
- Generate SRAM memories and compare genIP lists across memory releases to analyze area/timing/power (PPA) drift between IP releases, tracking bounding-box (bbox) area and bloat across compiled memories.
- Replaced a manual connectivity-extract → Excel → hand-drawn Visio diagramming workflow with an automated Verilog-parser → Postgres → FastAPI → React Flow pipeline, standing up the full chain end-to-end.

Team Leader — Battle Bots Design Team

Sep 2024 – Mar 2025

University of Toronto Robotics Association

Toronto, ON

- Led a team of 6 to 1st place (20+ teams) at UoT's Battle Bots Competition; designed a robust 3D-modeled chassis in AutoCAD, integrating IR and ultrasonic sensors for opponent tracking.
- Programmed hardware interrupts in Arduino C++ for deterministic sensor response, reducing detection error by 60%.
- Implemented PWM motor drivers and debugged high-speed spin stability, synchronizing actuator response with control logic across the hardware/software boundary under real-time constraints.

Projects

Embedded Gesture-Based Presentation Controller | STM32, ESP32, BLE HID, C

Mar 2026

- **Placed 3rd of 100+ course teams** with a wireless gesture controller built on an STM32F446ZE, an IMU, and an ESP32 BLE interface. Mapped IMU quaternion data to cursor motion and slide-change gestures over a UART MCU-to-MCU link. Analyzed end-to-end latency, bandwidth, and power tradeoffs across the sensing, firmware, wireless, and feedbacksystems.

Power Amplifier and Low Pass Filter PCB | LTSpice, Altium, PCB Assembly, Python

Jan 2025

- Designed a 40 V_{pp} Class D power amplifier with a 16 MHz LPF; compared BJT, comparator, and MOSFET topologies in LTSpice to handle high currents at elevated frequencies. Fabricated and tested a PCB, soldering and assembling RF-suitable components; validated performance using bench instruments and developed Python scripts for automated debugging.

RISC-V Soft-CPU Physics Visualizer | C, RISC-V Soft CPU, DE1-SoC FPGA, VGA, ModelSim

Mar 2025

- Deployed a RISC-V soft CPU on a DE1-SoC FPGA running particle-dynamics simulations written in C with VGA output. Met FPGA timing/memory constraints by optimizing computations with Taylor-series approximations. Validated behavior in ModelSim and on-board, tracing C-level performance down to RISC-V assembly, pipeline stalls, and memory limits.

Piano Tiles Game in Verilog | Quartus Prime, ModelSim, VGA, FSM Design

Dec 2024

- Implemented a Piano Tiles game in Verilog with keyboard input, speaker output, and VGA rendering using ROM for tile graphics and memory-mapped I/O for peripherals. Designed control logic through a finite state machine (FSM) for game progression, validated correctness and timing via ModelSim simulation, then deployed on FPGA hardware.